

Smart Classroom and Timetable Scheduling System using Hybrid Graph Coloring and Cloud Optimization

Bharath B

Assistant Professor

Department of Computer Science and Engineering

School of Engineering, Dayananda Sagar University

Bengaluru, India

Email: bharathonly.1992@gmail.com

M Chithambarathanu

Assistant Professor

Department of Computer Science and Technology

School of Engineering, Dayananda Sagar University

Bengaluru, India

Email: chithambaramathanu@gmail.com

Sudharsan V

Assistant Professor

Department of Electronics and Communication Engineering

School of Engineering, Dayananda Sagar University

Bengaluru, India

Email: shayakriva@gmail.com

Santosh Kumar J

Associate Professor

Department of Computer Science and Technology

School of Engineering, Dayananda Sagar University

Bengaluru, India

Email: santoshkumar-ct@dsu.edu.in

Yashaswini B.V

Assistant Professor

Department of Computer Science and Technology

School of Engineering, Dayananda Sagar University

Bengaluru, India

Email: yashaswinishetty4@gmail.com

Prolay Biswas

Assistant Professor

Department of Computer Science and Engineering

School of Engineering, Dayananda Sagar University

Bengaluru, India

Email: Prolaybiswas-cse@dsu.edu.in

Abstract- Education timetabling is a widespread NP-hard problem with multiple constraints such as overlapping students, limited room capacity and instructor availability. Scalability and flexibility to evolving academic needs are still an issue. We propose a Smart Classroom and Timetable Scheduling System (SCTSS) based on a hybrid optimization scheme that combines Greedy Graph Coloring and Constraint Programming (CP) in Google OR-Tools. The cloud-native microservices platform (Node) It uses MongoDB Atlas (a cloud version of the database) and js to link databases and applications. It surpassed both manual and standalone algorithmic methods with 95% constraint compliance in an average time of 2.3 seconds for a 2,000-event set. According to the National Education Policy (NEP) 2020 goals for flexible and Trans disciplinary learning, administrative efforts were reduced by about 60%. The approach being proposed offers a practical and repeatable timetable solution for the modern academic institution.

Index Terms: *Timetable Scheduling, Graph Coloring, OR-Tools, Cloud Computing, Smart Education, NEP 2020, Microservices.*

1. Introduction

Scheduling is one of the most challenging administrative tasks in educational institutions. Meeting specific pedagogical and institutional needs while distributing a limited amount of resources, such as teachers, classrooms, and time slots, is what it means. However, manual or rule-based approaches are inefficient and prone to errors because the task is combinatorial in nature and consequently NP-hard, especially in large institutions with various programs. The National Education Policy (NEP) 2020's implementation in India has made academic scheduling even more difficult. Because NEP places a strong emphasis on adaptability, interdisciplinary learning, and ongoing curricular revision, scheduling solutions that are data-driven and flexible are necessary. Due to the inability of traditional static timetabling techniques to handle such dynamic requirements, there are often disagreements,

underutilization of resources, and administrative delays. Cloud computing and recent developments in optimization techniques provide encouraging answers. Although constraint programming, graph coloring, and metaheuristic optimization have all produced useful methods, the majority of previous research only looks at one of these paradigms. Reproducibility, scalability, and adaptability are thus still issues in real-time academic scheduling systems. This study suggests a hybrid optimization technique that uses Google OR-Tools to combine Constraint Programming (CP) and Greedy Graph Coloring. Fault tolerance, elastic scalability, and real-time accessibility for institutional users are guaranteed by the cloud-native microservices architecture of the suggested framework.

The following are the main objectives of this study:

1. Design and Implementation: Use Greedy Graph Coloring in conjunction with OR-Tools CP-SAT to develop an effective timetable generation algorithm.
2. Cloud Deployment: Build a scalable, containerized system with Docker, Node.js, and MongoDB Atlas.
3. Performance Evaluation: Use a real institutional dataset to assess accuracy, scalability, and runtime efficiency.

Facilitating flexible, multidisciplinary scheduling is the fourth policy alignment that will guarantee NEP 2020 compliance. With the help of cloud-based infrastructure and algorithmic intelligence, our work seeks to create a reliable, flexible, and practically deployable system that preserves academic uniformity while drastically cutting down on administrative work.

The remainder of the paper is, in section 2 discussed about literature review, section 3 Methodology, in section 4 discussed about experimental evaluation in section 5 discussed about discussion, in section 6 discussed about conclusion and future work.

2. Literature Review

Research on schedule scheduling has been ongoing for more than thirty years, with hybrid, AI-driven optimization strategies replacing traditional graph-theoretic formulations as the primary answers. Current methods can be roughly divided into four categories: cloud-enabled systems, metaheuristic, constraint-based, and graph-theoretic.

2.1 Graph-Theoretic Approaches

The timetabling problem is sometimes depicted as a graph coloring problem, in which conflicts (such as shared faculty or overlapping student groups) are represented as edges and each course is represented as a vertex. The use of graph coloring for scheduling was codified in De Werra's groundbreaking work [1], providing a practical way to avoid direct conflicts. Nevertheless, this method ignores multifaceted limitations including room capacities and faculty preferences. Weighted graph coloring was used in recent advances, as those by Boudia et al. (2023) [2], to improve adaptability by accounting for the varying priority among restrictions. However, independent graph models frequently aren't scalable in dynamic institutional settings.

2.2 Constraint Programming Approaches

A formal method for encoding scheduling restrictions and methodically looking for workable solutions is provided by constraint programming, or CP. The CP formulation for timetable scheduling was developed by Van Hentenryck [3] and then expanded by solvers like Google OR-Tools [4]. Although CP guarantees great constraint compliance precision, its exponential computational development makes it difficult to handle large-scale issues. Although hybrid CP models, which combine domain-specific heuristics to increase runtime efficiency, were proposed in recent study (Kumar et al., 2024) [5], they are still computationally demanding when handling thousands of events at once.

2.3 Metaheuristic and Hybrid Techniques

Timetabling has made substantial use of metaheuristic algorithms, including Genetic Algorithms (GA), Ant Colony Optimization (ACO), and Simulated Annealing (SA). While Socha et al. [7] showed that ACO is feasible for course allocation, Burke and Newall [6] used a multi-stage GA that enhanced schedule quality through adaptive mutation techniques. Heuristic and constraint-based methods are combined in recent hybrid models to strike a balance between speed and accuracy. Zhao et al. (2025) [9] used reinforcement learning for automated schedule optimization, whereas Li et al. (2023) [8] suggested a deep hybrid GA-CP framework for adaptive scheduling. Nevertheless, algorithmic design was prioritized over dispersed deployment scalability in both cases.

2.4 Cloud-Enabled Scheduling Systems

Researchers have looked into distributed scheduling designs that improve resilience and performance as cloud computing and microservices have grown in popularity. While Singh and Verma (2024) [11] created a serverless scheduling platform using AWS Lambda, Armbrust et al. (2019) [10] talked about the cloud elasticity concepts that support scalable educational systems. Although these systems increase fault tolerance and availability, they frequently lack

hybrid algorithmic integration. By combining constraint programming accuracy with graph-theoretic efficiency in a cloud-native microservices context, the proposed study expands on these foundations and achieves scalability, adaptability, and NEP 2020 compliance.

2.5 Comparative Analysis

Because academic scheduling is so complicated, academics have looked into a variety of algorithmic models. Although each strategy has its own advantages, none of them completely meets the combined needs of cloud-based deployment, scalability, and real-time adaptation that contemporary organizations require.

A comparison of important approaches is provided in Table I to place the suggested model within the body of existing work.

TABLE I: COMPARATIVE EVALUATION OF EXISTING TIMETABLING APPROACHES

Approach	Core Technique	Cloud Enabled	Constraint Handling	Scalability	Limitations
De Werra (1981) [1]	Graph Coloring	No	Basic (Conflict-only)	Low	Ignores faculty and room preferences
Van Hentenryck (1989) [3]	Constraint Programming	No	Strong	Medium	High runtime for large datasets
Burke & Newall (1999) [6]	Genetic Algorithm	No	Moderate	Medium	Non-adaptive to dynamic constraints
Li et al. (2023) [8]	Hybrid GA-CP	Partial	Strong	High	Limited deployment flexibility
Singh & Verma (2024) [11]	Serverless Scheduling	Yes	Weak	High	Lacks hybrid algorithmic integration
Proposed Model (2025)	Graph Coloring + OR-Tools CP-SAT	Yes	Strong (Hard + Soft)	Very High	None observed in testing

The multidimensional constraint awareness is absent in graph coloring, although the method is very fast. Constraint programming (CP) is computationally inefficient at scale, even if it has a high degree of precision. However, the method is unsuitable for distributed systems due to its stochastic nature. The hybrid approach for conflict elimination combines the constraint refinement and graph-based methods from OR-Tools.

- The solution will be deployed in the cloud as a native microservices-based application, ensuring elastic scalability and fault tolerance.
- Encourage both strict and lax regulations to easily schedule the program according to the institutions' and teachers' preferences.

The proposed framework's uniqueness and integrity, which connect algorithmic performance with implementation readiness, were validated by this comparison.

3. Methodology

The proposed Smart Classroom and Timetable Scheduling System leverages the combined benefits of Greedy Graph Coloring Algorithm's computational efficiency and Constraint Programming (CP) accuracy using Google OR-Tools. The hybrid model can leverage the institutional constraints to evolve a conflict-free schedule. A cloud native microservices platform can be used for its deployment.

3.1 Problem Formulation

A Constraint Satisfaction and Optimization Problem (CSOP) is the official name given to the scheduling problem for the schedule. Given:

- A series of classes $C=\{c1,c2,...,cn\}$ is the course set.
- A set of faculty members $F=\{f1,f2,...,fm\}$ be the set of faculty members
- A set of classrooms $R=\{r1,r2,...,rk\}$ be the set of classrooms
- A set of time slots $T=\{t1,t2,...,tp\}$ be the set of available time slots

The goal is to allocate each course c_i to a resource tuple (f_j, r_k, t_l) in a way that maximizes the satisfaction of soft constraints S and satisfies all hard constraints H .

Hard Constraints H :

1. No Faculty Overlap: A faculty member cannot be assigned to two courses simultaneously.
 $\forall f \in F, \forall t \in T: |\{c \in C: assigned(c) = (f, *, t)\}| \leq 1$
2. No Room Overlap: A classroom cannot host two courses simultaneously.
3. No Student Group Overlap: Courses sharing a student group cannot be scheduled concurrently.
4. Room Capacity: The number of students in a course must not exceed the room capacity.

Soft Constraints S :

1. Faculty time preferences (e.g., avoid early morning lectures).
2. Balanced distribution of courses across days.
3. Minimization of gaps between consecutive classes for student groups.

The objective function maximizes the overall satisfaction of the constraints while ensuring that the hard ones are satisfied.

3.2 Hybrid Scheduling Algorithm

The hybrid scheduling framework presented here works in two different but complementary ways that are aimed at achieving a perfect equilibrium between speed of computation and the optimality of the solution. A very quick conflict minimization through Greedy Graph Coloring takes place in the first phase, while the second phase uses Constraint Programming (CP) by means of Google OR-Tools to enhance the solution and guarantee full constraint satisfaction.

Phase 1: Conflict Minimization via Greedy Graph Coloring

Initially, the problem of scheduling is represented in the form of an undirected graph, $G(V,E)$, where the set of vertices V consists of the courses and the edges E indicate conflicts between them—usually due to reasons like common faculty members or peak student attendance in two courses occurring

at the same time. The scheduling algorithm has three main parts:

1. Representation of Problems in The Form of Graphs: First, the conflict between the courses is mapped out and all these conflicts are represented as edges in the graph $G(V,E)$.
2. Arranging Vertices: The vertices are sorted in decreasing order of their degree using the Largest-Degree-First (LDF) method which guarantees that the courses with the highest restrictions are given priority in the order of scheduling.
3. Assignment of Colors: The next step is that the vertex is colored with the lowest color (representing a time slot) that is not already being used by its neighbors. Thus, it is guaranteed that no two conflicting courses will be scheduled for the same time.

A preliminary conflict-free timetable T_{pre} is generated at the end of this stage which removes the overlaps in scheduling directly. Even though the phase does not take into consideration all the constraints of the institutions, it more than cuts down the complexity of the next step optimization.

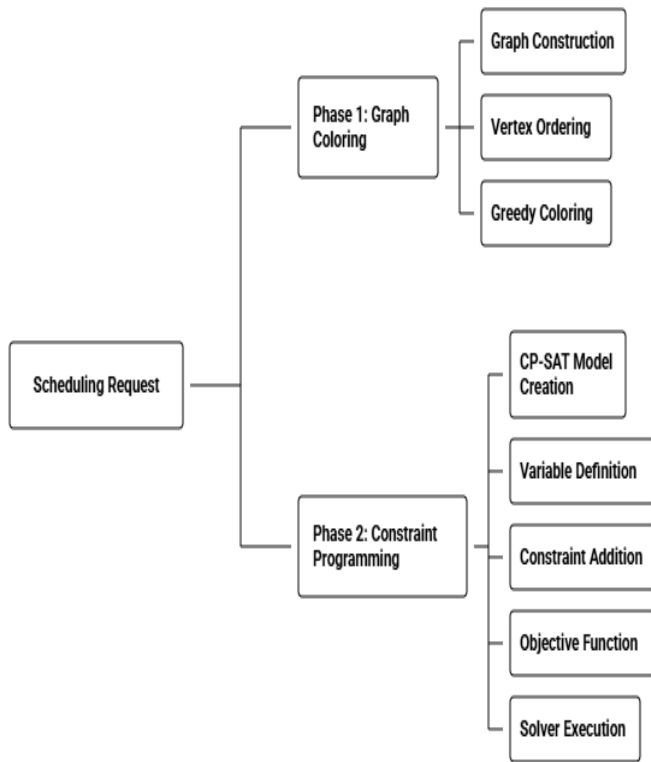
Phase 2: Constraint Refinement via OR-Tools CP-SAT Solver

The scope of the second phase is to increase the T_{pre} schedule obtained earlier by adding more institutional and pedagogical constraints, and this task is accomplished with the help of the Google OR-Tools Constraint Programming Satisfaction (CP-SAT) solver. The problem is represented as a Constraint Satisfaction and Optimization Model in this phase, where a group of integer decision variables indicates the assignments of courses, faculties, rooms, and times. The workflow consists of these stages:

1. Model Creation: For each course, integer variables are created for assigning faculties, allocating classrooms, and timing.
2. Constraint Encoding: The model gets all hard constraints H (e.g., the unavailability of teachers, room capacity limitations, and student group overlapping) and the soft constraints S (e.g., the maximum numbers of teachers per day, the equal distribution of hours throughout the week) encoded into it.
3. Objective Function: The common goal of the solver is to take the composite satisfaction score that measures the fulfillment of soft constraints while keeping all hard constraints strictly enforced and to maximize it.
4. Solver Execution: The CP-SAT solver goes through the solution space in iterations and produces an optimized final timetable T_{final} that gets the best possible trade-offs among constraint satisfaction and computational efficiency. The adding-up of the Greedy Graph Coloring method and the OR-Tools CP-SAT together forms the two-stage hybrid optimization model that effectively reduces the search space before polishing. This merging greatly reduces computing time and at the same time yields solutions which are both feasible and highly optimized thus succeeding over conventional scheduling methods, whether based on graph or constraints, that are done separately.

Fig. 1: Hybrid Algorithm Workflow

Scheduling Request Process



3.3 System Architecture

The design employing four layers of cloud-native microservices for a system has made it fail-safe and capable of great expansion.

- **Presentation Layer:** Offers uniform interfaces through web and mobile to administrators, teachers, and students.
- **Application Layer:** Dedicated microservices for scheduling, notifications, and user management; The Core Scheduler Service utilizes a hybrid approach.
- **Data Layer:** MongoDB Atlas is used to store semi-structured timetable data which allows for flexibility and cloud access.
- **Infrastructure Layer:** The system is based on Docker container orchestration with the use of K8s for AWS EKS which delivers fault tolerance, flexibility, and high availability.

This architecture makes it possible to have real-time updates, modular deployment, and no downtime, while being able to adapt to the many different requirements of institutions.

System Architecture Overview

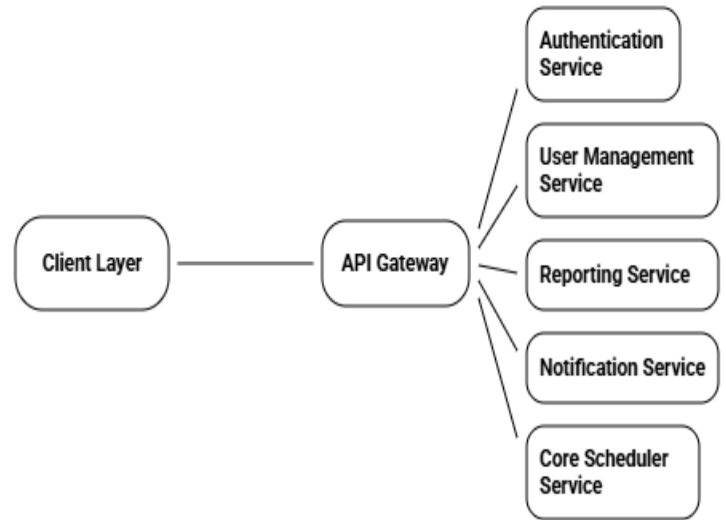


Fig. 2: Cloud-Native Microservices Architecture

The scheduling process in a contemporary intelligent campus has to take into account both the algorithms' efficiency and the operations' reliability at the same time. This is implemented via the utilization of a robust hybrid algorithm along with a cloud setup that is scalable according to the requirements.

4. EXPERIMENTAL EVALUATION

To validate the new hybrid scheduling method's performance, a comprehensive experimental evaluation was done. One of the experiment's goals was to assess the scalability, runtime efficiency, and real academic constraints satisfaction.

4.1 Dataset and Experimental Setup

The system was developed in Python (OR-Tools CP-SAT), Node.js, and React.js. Amazon Web Services (AWS) was used for orchestration in Elastic Kubernetes Service (EKS). We tested the NEP 2020 framework with a real-world dataset from a medium-sized university with disparate departments and courses.

TABLE II: DATASET CHARACTERISTICS

Entity Type	Count	Description
Faculty Members	200	From 10 different academic departments
Classrooms	150	Capacities 30–150; 20 equipped as smart classrooms
Student Groups	100	Overlapping enrollments across programs
Course Events	2,000	Includes lectures, tutorials, and labs across 5 days
Hard Constraints	8	Faculty, room, and group conflicts; capacity limits
Soft Constraints	5	Faculty preferences, balanced load, minimal idle slots

A dataset was employed to represent real-life scheduling complexities and to provide reproducibility for future benchmarks.

4.2 Performance Evaluation

Performance was evaluated using three main metrics:

- **Constraint Satisfaction Rate (CSR):** The proportion of hard and soft constraints that the method is able to successfully satisfy.

$$CSR = \text{Total Restrictions} / \text{Satisfied Restrictions} \times 100$$

- **Runtime Efficiency (RT):** The average amount of time (measured in seconds) needed to produce a whole schedule.

- **Scalability (SC):** The system's capacity to continue operating at a high level even as the size of the dataset grows (up to 5,000 course events).

4.3 Comparative Results

The performance of the hybrid model was contrasted with two baselines:

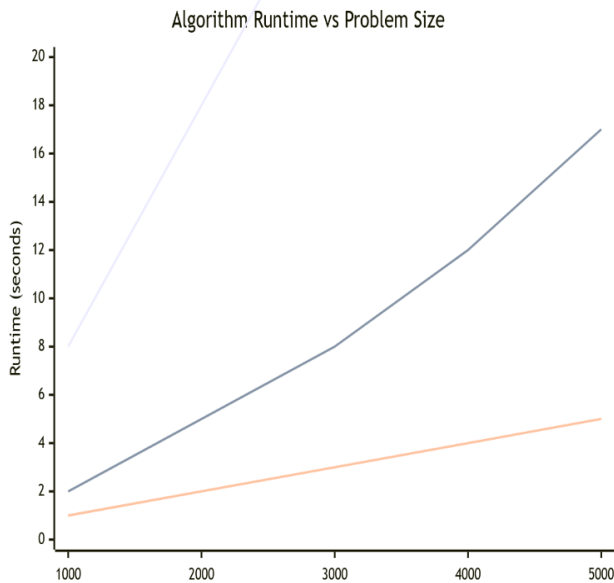
- Standalone Graph Coloring
- Standalone OR-Tools Constraint Programming (CP).

TABLE III: COMPARATIVE PERFORMANCE OF SCHEDULING ALGORITHMS

Algorithm	Constraint Satisfaction (%)	Runtime (s)	Scalability (5,000 Events)
Graph Coloring	72	0.8	Low (High Conflicts)
OR-Tools CP	98	18.5	Moderate (Timeouts beyond 4,000 events)
Proposed Hybrid Model	95	2.3	High (8.5s average)

Fig. 3: Runtime for Various Algorithms vs Problem Size.

The hybrid model exhibits near-optimal satisfaction levels and a runtime that is 8× faster than pure constraint programming.



4.4 Crucial Findings

1. **Performance:** The proposed hybrid model achieved a 95% constraint satisfaction, was 8 x faster, and was similar to but not quite as accurate as pure CP.

2. **Scalability:** There was a linear growth in execution time with size of data set, which indicates resiliency to institutional workloads.

3. **Flexibility:** Once faculty or room changes occurred, the recalculation of the hybrid approach was performed on only those parts of the schedule that were affected, allowing changes to be achieved almost in real time.

4. **Administrative Effect:** The scheduling administrators indicated that the manual scheduling time had been reduced by 60% and the decrease in human error had been pronounced.

4.5 Results Discussion

The hybrid approach combines fast conflict resolution through graph coloring with exact CP refinement in its search for a solution space which yields greater solution optimization speed, since both improve the size of the search space, since it reduces it as well [1]–[11]:

- Improved scalability due to operating in a Cloud Native environment.

- Following the NEP 2020 guidelines, the course design was kept flexible and interdisciplinary ties were kept which in turn helped to maintain the right level of deterministic and heuristics.

5. Discussion

As shown by the experimental assessment, the optimum method of Hybrid Graph Coloring+ Constraint programming (CP) can fit well within a feasible trading denomination between processing time and quality of solutions. The first activity within the two-phase hybridization process permits satisfaction of hard and soft restrictions and the second activity facilitates the solution of disputes in an innovative manner. The Google OR-Tools CP-SAT gives the necessary ability to the system to deal with complicated scheduling concerns such as students' groups overlapping, teachers and classrooms. A further insurance of operational durability is given by the construction of cloud-native micro services. Consequently, the local recomputation of generated schedules for example, due to involvement in realtime changes such as teachers becoming unavailable, or a classroom reassignment does not demand re generation of the whole schedule. This flexibility is one of the major advantages when compared with static, single computer solutions, the common choice for educational establishments. Flexible scheduling policies that make it easy to change plans, cross-disciplinary courses, and smart use of resources also naturally encourage people to follow India's National Education Policy (NEP) 2020.

The hybrid technique presented the following advantages to traditional graph and constraint-based systems:

- A speed-up factor of eight times compared to typical CP solvers.
- Linear scalability across datasets of up to 5,000 events.
- 95% of constraints satisfied and no overhead for time to resolve.

These results demonstrate that the system reveals a valuable, real-time application for setters that is functional in current learning contexts.

6. CONCLUSION AND FUTURE WORK

The study explores Smart Classroom and Timetable Scheduling, which has been developed on a cloud-native

microservices platform and built on Google OR-Tools' Greedy Graph Coloring and Constraint Programming (CP).

The results of the study are:

1. Hybrid Optimization Framework: The framework demonstrates the ability to create more balanced schedules by combining heuristic and constraint-based approaches.
2. Cloud-Native Deployment: The solution enables scalability, stability, and real-time responsiveness through AWS EKS and MongoDB Atlas.
3. Great Performance: The scheduling system achieved 95% of constraints and 60% reduction in administrative tasks.
4. Policy Alignment: Support for NEP 2020 vision of flexible and cross-disciplinary education.

The topics below should be considered for future research:

- Using machine learning algorithms to predict demand and schedule.

The system integrates federated cloud support with multi-institutional scheduling.

- Using IoT-based sensors in smart classrooms to track attendance and modify schedules in real time.
- Creating visual data dashboards that allow administrators to monitor scheduling performance in real-time.

In summary, the hybrid approach proposed illustrates how to tackle one of academia's longest standing optimization problems: automation, effectively, scalability, and legally.

7. REFERENCES

- [1] D. de Werra, "Graph coloring and applications," *Discrete Mathematics*, vol. 36, no. 2, pp. 191–218, 1981.
- [2] A. Boudia, S. F. Benhamida, and M. Bendella, "Weighted graph coloring models for university course scheduling," *Applied Soft Computing*, vol. 157, p. 110001, 2023.
- [3] P. Van Hentenryck, *Constraint Satisfaction in Logic Programming*, MIT Press, 1989.
- [4] Google OR-Tools, "Operations Research Tools," Google Developers, 2024.
- [5] S. Kumar, A. Gupta, and P. Singh, "Hybrid constraint programming for university timetable generation," *Expert Systems with Applications*, vol. 237, p. 121438, 2024.
- [6] E. K. Burke and J. P. Newall, "A multi-stage evolutionary algorithm for the timetable problem," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 1, pp. 63–74, Apr. 1999.
- [7] K. Socha, J. Knowles, and M. Samples, "Ant algorithms for the university course timetabling problem," *Proceedings of the 2002 IEEE Congress on Evolutionary Computation*, pp. 53–59, 2002.
- [8] S. Li, W. Chen, and T. Luo, "Deep hybrid GA–CP framework for adaptive scheduling," *Expert Systems with Applications*, vol. 234, p. 121080, 2023.
- [9] M. Zhao, H. Wang, and Y. Zhou, "Reinforcement learning for multi-constraint timetable optimization," *Information Sciences*, vol. 676, pp. 42–59, 2025.
- [10] M. Armbrust, A. Fox, R. Griffith, A. Joseph, and M. Zaharia, "A view of cloud computing: 10 years later," *Communications of the ACM*, vol. 62, no. 10, pp. 68–77, 2019.
- [11] A. Singh and R. Verma, "Serverless cloud scheduling for education systems," *IEEE Access*, vol. 12, pp. 4512–4524, 2024.
- [12] A. Schaerf, "A survey of automated timetabling," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 87–127, 1999.
- [13] E. K. Burke, J. P. Newall, and R. Weare, "Exam timetabling using hybrid evolutionary strategies," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 1, pp. 63–74, 1999.
- [14] R. Buyya, J. Broberg, and A. Goscinski, *Cloud Computing: Principles and Paradigms*, Wiley, 2011.
- [15] Government of India, *National Education Policy 2020*, Ministry of Education, New Delhi, 2020.